

Experiment 5**Study of Addressing Modes by Analyzing the Execution of a C Program using GNU Debugger (GDB)**

Experiment No.	5	Subject	COA
Branch	CSE	Name	Anshul Modi
Roll No.	241210021		

1. Aim / Objective

To study different addressing modes of instructions by analyzing the execution of a C program step-by-step using the GNU Debugger (GDB). Eight addressing modes — Immediate, Direct, Data Segment (Global), Indirect, Indexed, Base Register, Relative, and Auto Increment — are demonstrated in a single unified C program.

2. Theory

An addressing mode specifies how the operand of an instruction is determined. Modern processors support multiple addressing modes to provide flexibility in accessing data from registers, immediate constants, memory locations, and pointers. Understanding addressing modes is fundamental to computer architecture as they directly influence instruction encoding, program efficiency, and memory access patterns.

Addressing Mode	Code Line	Description
Immediate	result = a + 5	Constant 5 is embedded directly in the instruction
Direct	result = a + b	Operands accessed directly from named memory locations
Data Segment	result = x	Global variable x accessed via RIP-relative address (x86-64)
Indirect	ptr = &a; result = *ptr	Address of 'a' stored in ptr; value obtained by dereferencing
Indexed	result = arr[1]	Effective address = base address of arr + index × element size
Base Register	result = *(base + 2)	base pointer + constant offset used to compute effective address
Relative	goto label	Control transferred to a label relative to the current instruction pointer
Auto Increment	result = *ptr++	Pointer dereferenced then incremented to point to next element

GDB allows us to step through a C program line by line, print variable values, inspect memory addresses, view CPU registers, and disassemble machine code — making it an ideal tool to observe how each addressing mode operates at the hardware level.

3. Source Code — lab5_addressing_modes.c

A single C program demonstrating all eight addressing modes:

```
#include <stdio.h>

int x = 10;

int main(){
    int a=10, b=20;
    int result;
    int arr[3]={1,2,3};
    int *ptr;
```

```

// Existing Modes
result = a + 5;      // Immediate
result = a + b;      // Direct
result = x;          // Global/Data segment

ptr = &a;
result = *ptr;        // Indirect

result = arr[1];      // Indexed

// 7. Base Register Addressing
int *base = arr;
result = *(base + 2);

// 8. Relative Addressing
goto label;

result = 999;

label:
    result = result + 1;

// 11. Auto Increment Addressing
ptr = arr;

result = *ptr++;

printf("Result : %d\n", result);

return 0;
}

```

Fig. 1 – Source code of lab5_addressing_modes.c with all addressing modes annotated.

4. GDB Commands Used

GDB Command	Purpose
gcc -g lab5_addressing_modes.c -o out1	Compile with debug symbols
gdb ./out1	Launch GDB
break main	Set breakpoint at main
run	Start execution
next	Step to next source line
print a / print b / print x	Inspect variable values
print &a	Print memory address (Direct)
print ptr / print *ptr	Inspect pointer (Indirect)
print arr[1]	Inspect array element (Indexed)
print *(base+2)	Inspect base register result
print result	Verify relative / auto-increment result
info registers	View all CPU register values
disassemble main	View assembly for main function

5. Procedure

1. Write the combined C program (lab5_addressing_modes.c) covering all eight addressing modes.

2. Compile with debug symbols: `gcc -g lab5_addressing_modes.c -o out1`
3. Launch GDB: `gdb ./out1`
4. Set a breakpoint at main: `break main`, then start with `run`.
5. Use `next` to step through each line one at a time.
6. After `result = a + 5`, use `print result` to confirm Immediate addressing (result = 15).
7. After `result = a + b`, use `print result` to confirm Direct addressing (result = 30).
8. After `result = x`, use `print result` to confirm Data Segment access (result = 10).
9. Use `print &a` to view the memory address, then `print ptr` and `print *ptr` to verify Indirect addressing.
10. Use `print arr[1]` to confirm Indexed addressing (value = 2).
11. After `result = *(base + 2)`, use `print result` to confirm Base Register addressing (value = 3).
12. Step through `goto label` and verify that `result = 999` is skipped (Relative addressing).
13. After `result = *ptr++`, verify Auto Increment: `result = arr[0] = 1`, `ptr` advanced to `arr[1]`.
14. Use `info registers` to view all CPU register values (`rax`, `rsp`, `rip`, `eflags`, etc.).
15. Use `disassemble main` to view the x86-64 assembly and correlate each C line with its addressing mode.

6. GDB Session Screenshots

Fig. 2 – GDB session: compilation, breakpoint at main, stepping through lines, printing variable values (`a=10`) and memory address of `'a'` (`0x7ffffffdd54`).

Fig. 3 – GDB continued: pointer inspection (`ptr`, `*ptr`), `arr[1]=2` via Indexed addressing, `*(base+2)=3` via Base Register, `goto label` skipping `result=999` (Relative), `*ptr++=1` with pointer advanced (Auto Increment), full step-through of all lines, and 'info registers' showing `rax`, `rsp`, `rip`, `eflags` and segment registers.

Fig. 4 – 'disassemble main' output: x86-64 assembly confirming Immediate (`add $0x5`), Direct (`mov` via `rbp` offsets), Indirect (`lea + mov` via pointer), Indexed (`mov` with scaled index), Base Register (`lea + offset`), Relative (`jmp` to label), Auto Increment (`mov + inc`), and RIP-relative access for global variable `x`.

7. Observations

- **Immediate Addressing:** `result = a + 5` compiled to `add $0x5, %eax` — the constant 5 is part of the instruction itself.
- **Direct Addressing:** `result = a + b` used `MOV` instructions with base-pointer offsets to fetch both variables from the stack.
- **Data Segment (Global):** `result = x` used a RIP-relative `MOV` instruction to access the global variable from the data segment.
- **Indirect Addressing:** `ptr = &a` stored the address `0x7ffffffdd54` in `ptr`. `*ptr` dereferenced it to yield 10.
- **Indexed Addressing:** `result = arr[1]` accessed memory at `arr`'s base address + 4 bytes (`1 × sizeof(int)`). GDB confirmed value = 2.
- **Base Register Addressing:** `result = *(base + 2)` computed the effective address as base pointer + `2 × sizeof(int)`. GDB confirmed value = `arr[2] = 3`.
- **Relative Addressing:** `goto label` emitted a near-jump instruction (`jmp`) that transfers control relative to RIP, causing `result = 999` to be skipped entirely.
- **Auto Increment Addressing:** `result = *ptr++` dereferenced `ptr` (yielding `arr[0] = 1`) and then incremented `ptr` to point to `arr[1]` in a single operation.
- **Registers:** `rax` held computed results, `rsp` pointed to the stack frame, `rip` held the next instruction address, and `eflags` reflected arithmetic flags.
- The single combined program allowed all eight modes to be verified in one continuous GDB session.

8. Result

All eight addressing modes — Immediate, Direct, Data Segment (Global), Indirect, Indexed, Base Register, Relative, and Auto Increment — were successfully demonstrated in a single C program and verified using GDB. The disassembled output confirmed how each C statement maps to a specific x86-64 assembly addressing mode at the machine level.